

UNO 终端卡牌游戏

实验指导手册

目录

1	实验概述	3
1.1	你要做一个什么样的程序	3
1.2	三个阶段	3
1.3	提交	3
2	准备工作	3
2.1	目录结构	3
2.2	构建与运行	3
3	阶段一：会话与大厅	4
3.1	要实现什么	4
4	阶段二：基础规则	4
4.1	要实现什么	4
4.2	牌堆与确定性洗牌	4
4.3	确定性牌堆（测试用）	5
4.4	消息分发	5
5	阶段三：拓展规则	5
5.1	要实现什么	5
5.2	消息形状	6
5.3	需要显式建模的中间状态	7
6	注意事项	7
7	局域网联机	7
8	消息速查	8

1 实验概述

1.1 你要做一个什么样的程序

一个运行在终端的卡牌游戏 UNO。游戏程序分为两个模式，Host 模式作为房主维持游戏房间，并负责运行主要的游戏逻辑；Client 模式可以加入已有的房间进行游戏。

1.2 三个阶段

阶段	任务
阶段一	会话与大厅：玩家身份、加入/离开、名单同步、开局条件
阶段二	基础规则：牌堆、发牌、回合、匹配、功能牌、计分
阶段三	拓展规则：+4 质疑、叠加、0-7 换牌、同牌叠出、UNO 喊牌、牌堆耗尽、双人规则

1.3 提交

提交分为线上提交和线下验收两部分，线上提交占 10% 分数，线下验收占 90% 分数。

1. 能通过默认测试与评分测试的源码，打包成 .zip 格式的压缩包在平台上提交，以最终提交为准。助教会根据代码设计进行评分。
2. 国庆假期后的第一次实验课进行第 0 次线下验收，不计分；之后每两周进行一轮计分验收，依次验收三个阶段。

如果未能在指定轮次进行对应阶段的验收，之后每迟一轮，扣除该阶段 20% 的分数。计分验收共有五轮，之后将不再进行验收，没有进行验收的部分以零分计算。

2 准备工作

2.1 目录结构

src/protocol/	行式协议、事件循环、本地传输、TCP 传输（框架，不要改）
src/game/	会话层：命令表、host/client 的事件钩子（你要实现的部分）
src/basic/	小工具（框架）
tests/	默认测试与评分测试
docs/	本手册、规则与协议（Typst 源码与编译出的 PDF）
run-tests.sh	测试入口（见下一节）

2.2 构建与运行

```
cmake -S . -B build
cmake --build build -j
```

运行游戏（主机开房间，客户端加入，见 --help）：

```
./build/uno --room=demo --name=Host --seed=42
./build/uno --join --room=demo --name=Alice
```

对应的自动化测试脚本：

```
./run-tests.sh          # 全部（框架回归 + 三个阶段）
./run-tests.sh 1        # 阶段一：会话与大厅
./run-tests.sh 2        # 阶段二：基础规则
./run-tests.sh 3        # 阶段三：拓展规则
```

每次运行都会先跑一遍框架回归，再跑该阶段的测试；跑完会打印每个评分测试的分数。阶段是累进的：阶段二、阶段三的测试在阶段一没完成时会失败。

3 阶段一：会话与大厅

3.1 要实现什么

1. 让客户端在连上之后发送 JOIN (`ClientSession::on_connected()`)。
2. 在主机收到 JOIN 时分配座位、回 WELCOME，并向所有人广播 PLAYERS 消息。
3. 处理 RENAME 消息与主机的 kick 命令：改完名单都要广播新的 PLAYERS。
4. 在玩家掉线时 (`on_peer_leave`) 把他从名单里清掉，并广播新的 PLAYERS。
5. 在 start 里检查开局条件：至少两名玩家，否则报错并留在大厅。
6. 主机的 start 命令在开局成功时必须调用 `enter_game()`，否则主机的命令表不会切换到游戏阶段。客户端收到 START 时由框架切换。

4 阶段二：基础规则

4.1 要实现什么

1. 构造 108 张牌组，并按下一节的顺序用 seed 洗牌，让同一个 `--seed` 在任何实现下都发出同一手牌。
2. 在 start 里发牌：每人 7 张，翻出引牌；先给每人发送私有的 HAND，再广播 START 与 TURN。引牌是功能牌时按规则生效。
3. 判定出牌是否合法：与当前牌面的颜色相同或图案相同；w/w4 必须指定颜色。
4. 处理抽牌与 pass：无牌可出时抽 1 张；抽牌后本轮不能再出原有手牌。
5. 实现功能牌效果：SK、RV、D2、W（细则见规则文档）。
6. 有人出完手牌时结算分值、广播 GAMEOVER，然后回到大厅。
7. 维护消息分发规则：HAND 只发给本人，公开信息广播给所有人（见下表）。

4.2 牌堆与确定性洗牌

牌组构成见规则文档的「牌组」一节；顺序与洗牌方式是本作业测试接口，请严格遵守，否则评分器无法核对同一 `--seed` 下的手牌：

1. 牌堆构造顺序固定如下：

- 颜色按 R、Y、G、B 的顺序；
- 每种颜色内按 0, 1, 1, 2, 2, ..., 9, 9, SK, SK, RV, RV, D2, D2 的顺序；
- 然后是 W 四张、W4 四张；

- 合计 $4 \times (1 + 18 + 6) + 4 + 4 = 108$ 张。

2. **洗牌**: 用 seed 初始化 `std::mt19937`, 然后 `std::shuffle(deck.begin(), deck.end(), gen)`。
3. **发牌**: 从洗好的牌堆头部开始, 座位 0 拿 7 张, 座位 1 拿接下来 7 张, 以此类推; 再取下一张作为引牌, 其余留在牌堆中。
4. **未提供 seed 时**可以用随机种子 (例如 `std::random_device`); 一旦提供了 `--seed`, 整局必须完全可复现。

4.3 确定性牌堆 (测试用)

`--seed` 只能复现随机发出来的一副牌, 没法构造「某人手里正好有一张 +4」这种局面。框架为此提供了牌堆覆盖:

```
uno --room=demo --deck=R5,R7,GD2,W4,...
```

对应 `Session::set_deck_override() / has_deck_override() / deck_override()`, 契约如下:

- 给了覆盖就不洗牌: 严格按给定顺序发牌, 座位 0 拿前 7 张, 座位 1 拿接下来 7 张, 以此类推; 下一张作为引牌; 其余按顺序作为摸牌堆。
- 牌堆可以比 108 张短, 也可以更长; 摸完之后按正常的「保留牌面、弃牌堆重洗」规则继续。
- 长度不足 $7 \times \text{人数} + 1$ 时, `start` 必须报 `ERROR` 并且不开局。
- 给了覆盖就忽略 `--seed`。

评分测试就是用这个接缝摆局面的, 你自己验证边界情况时也可以用。

4.4 消息分发

消息	发给谁	原因
HAND	本人	手牌是私有信息
WELCOME	新玩家	座位号只对他有意义
PLAYERS	所有人	名单是公共信息
TURN / PLAYED / DRAWN	所有人	公开的牌桌状态
ERROR	相关玩家	非法操作只对他有意义

5 阶段三: 拓展规则

5.1 要实现什么

1. **W4 的合法性判断与质疑窗口**: 不质疑、质疑不成立、质疑成立三种结果都要实现 (消息形状见下)。
2. **罚抽链**: D2 之间可以叠加, W4 可以叠在罚抽牌上; **TURN** 要带 `pending=`。
3. **0 与 7 的换牌**: 全体交换与指定对象交换, 相关玩家各收到新的私有 **HAND**。
4. **多张出牌**: 同色同数字、同数不同色 3 张及以上、同色 SK 叠出。
5. **UNO 与举报**: 手里正好剩 1 张时喊牌才合法, 其它张数喊都算错喊; 漏喊在下家行动之前可以举报。两种情况都罚抽 2 张。

6. 牌堆耗尽时保留牌面，把弃牌堆重新洗成新牌堆。
7. 双人规则：Reverse 等同 Skip。
8. 所有等待窗口（选色、质疑、换牌、叠加、举报）里收到的其它命令都要拒绝，且不改变任何状态。

5.2 消息形状

拓展规则用到的消息形状比基础规则细，评分测试按下面这些字段断言。

5.2.1 多张出牌

PLAY 的 cards 可以用逗号分隔的多张牌，合法组合只有三种：

组合	例子	额外要求
同色同数字，2 张	R5,R5	—
同数字不同色，3 张及以上	R5,Y5,G5	必须带 color 指定颜色
同色 SK，n 张	RSK,RSK	跳过 n 个玩家

其它组合一律 ERROR。多张出牌之后手里必须至少还留 1 张（最后一张牌只能单出），否则 ERROR。指定了颜色的出牌，PLAYED 里必须带 color。

5.2.2 结束一局

有人出完最后一张牌时广播 GAMEOVER winner=<座位> points=<分值>：

- points 是其余玩家剩余手牌的分值之和；
- 数字牌 0-9 计 0-9 分，SK、RV、D2 计 20 分，W、W4 计 50 分；
- 之后所有人回到大厅。

5.2.3 UNO 与举报

- UNO（不带 player）：手里正好 1 张算合法，广播 UNO player=<座位>；其它张数都算错喊，广播 UNOPENALTY player=<座位> count=2，并给该玩家发送新的私有 HAND（+2 张）。
- UNO player=<目标>：目标是刚出过牌、现在只剩 1 张、并且没有喊过的玩家时，广播 UNOPENALTY（同上）并给他发送私有 HAND（+2 张）。
- 举报窗口在目标的下家出牌或抽牌之后关闭；窗口之外或目标不是 1 张牌，一律 ERROR。

5.2.4 0 与 7

- 打出 0：广播 SWAP all=yes；所有玩家按当前出牌方向把手牌交给下一位（顺时针时座位 i 的牌给座位 i+1，最后一位给座位 0）；每位玩家收到新的私有 HAND；然后广播新的 TURN。
- 打出 7：广播 SWAP player=<出牌者> waiting=yes，并且暂时不广播 TURN；出牌者随后发送 SWAP player=<目标>，主机交换双方手牌、给两人各发一条私有 HAND、广播 SWAP player=<出牌者> with=<目标>，然后广播新的 TURN。窗口之内由别人发送 SWAP、或窗口之外发送 SWAP，一律 ERROR。

5.2.5 +4 质疑

合法性：出 W4 时，出牌者手里不能有与当前有效颜色相同的牌（数字牌或功能牌；W、W4 不算）。打出 W4 之后主机先广播 PLAYED，再广播询问，然后暂停（不广播 TURN）：

```
CHALLENGE player=<受害者> ask=w4 cards=4 color=<颜色>
```

受害者回答 CHALLENGE accept=yes|no 之后广播结果：

回答	结果	后续
no	result=draw4	受害者抽 4 张（私有 HAND），被跳过。
yes, W4 合法	result=draw6	受害者抽 6 张（私有 HAND），被跳过。
yes, W4 违规	result=illegal	出 W4 者抽 4 张（私有 HAND）；受害者成为当前玩家。

无论结果如何，W4 指定的颜色都继续生效。窗口之外发送 CHALLENGE 一律 ERROR。

5.2.6 叠加（罚抽链）

D2 或 W4 打出后，下家可以继续叠加，也可以吃下累计张数：

- TURN 的 pending=<n> 表示当前玩家面前累计的罚抽张数；
- 上家是 D2：下家可以出 D2 或 W4；
- 上家是 W4：下家只能出 W4，并且这条链里必须已经出现过 D2；
- 叠加时**不检查颜色**：能否叠加只看牌型；
- 当前玩家发送 DRAW 表示吃下罚抽：广播 DRAWN player=<座位> count=<n>，给他发送私有 HAND (+n 张)，然后跳过该玩家；
- 叠加出来的 W4 不再触发质疑询问；只有链条里第一个 W4 会询问。

5.3 需要显式建模的中间状态

中间状态	在等什么
等待选色	打出 Wild 的玩家指定下一个颜色
等待质疑	被罚抽 4 张的玩家回答 challenge yes no
等待换牌目标	打出 7 的玩家用 swap <player> 指定对象
等待叠加决定	下家决定继续叠 +2/+4 还是接受罚抽
等待 UNO 举报窗口	下家出牌之前，任何玩家都可以举报漏喊

在这个状态里收到的其它命令都必须被拒绝，且不能改变任何状态。

6 注意事项

- 不要自己写 select、poll、fork、线程或信号处理；
- 不要绕开 send_to / broadcast 直接操作文件描述符；
- 不要把牌堆、手牌之类的状态塞进 ClientSession 里。

7 局域网联机

项目里写好了完整可用的 TCP 传输：

```
# 主机以 TCP 监听
./build/uno --tcp=1145 --name=Host --seed=42

# 另一台机器（或本机）加入
./build/uno --host=192.168.1.10 --port=1145 --name=Alice
```

你会发现同一份 host/client 代码在 AF_UNIX 和 TCP 之上都跑得通，因为两者实现的是同一个 Connection 接口。

8 消息速查

方向	消息	要点
C→H	JOIN / RENAME	大厅身份
C→H	PLAY / DRAW / PASS / SWAP	出牌与抽牌
C→H	UNO / CHALLENGE	喊牌与质疑
H→C	WELCOME / PLAYERS	座位与名单
H→C	HAND	只发给本人
H→C	TURN / PLAYED / DRAWN	公开牌桌状态
H→C	UNO / UNOPENALTY / CHALLENGE	拓展规则反馈
H→C	GAMEOVER / ABORTED	一局结束

完整字段表见 docs/protocol.typ。